Procesamiento de Datos y Modelos Avanzados en Simulink

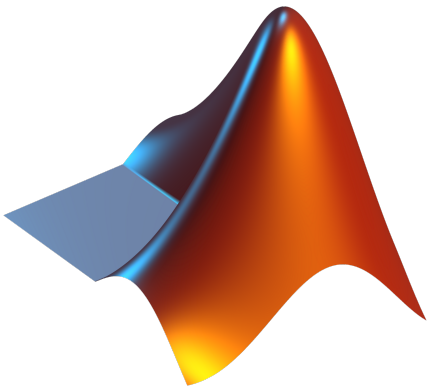
Fabrizio Benavidez[1]

https://github.com/Fabopy/Matlab-curso-

[1] *Facultad de Ciencias Exactas, Físicas y Naturales, Ingeniería Aeronáutica*

Resumen El siguiente trabajo consiste en el análisis de datos e imágenes con MATLAB. En el mismo se presenta el análisis de datos de un hoja de cálculo (Excel), así como el de una figura jpg.
Además, también se analiza otros modelos de Simulink un poco más avanzados. Se ven los bloques *From Workspace*, *LTI System* y *Fcn*

Palabras claves: Datos, imágenes, MATLAB, Simulink, bloques



19 de octubre de 2026

Semana 9

[1] fabrizio.benavidez@mi.unc.edu.ar

Guía 17: Adquisición y procesamiento de datos

Veamos la siguiente tabla hecha en Excel

	A	B	C	D
1				
2		3	C	
3		1hola9		
4		55	77	
5		numero 8		
6		1000	...	
7				
8				
9		4	6	
10				

Podemos extraer los datos de la misma y cargarla en MATLAB para procesar sus datos. El siguiente código muestra cómo hacerlo:

```
clc
clear all
close all


[num, txt, todo] = xlsread("Libro1.xlsx");
>> num son numeric
>> txt son strings
>> todo cell
vec = [];

for i=1:size(todo, 1)
    for j = 1:size(todo, 2)
        if ischar(todo{i, j})
            temp = todo{i, j};
            for k=1:length(temp)
                n = str2num(temp(k));
                if ~isempty(n)
                    fprintf('%g es un numero\n', n)
                    vec = [vec, n];
                end
            end
        end
    end
end

vec;

>> 1  9  8
```

El mismo código muestra cómo extraer los números de los datos que son strings o cadenas de textos.

Ahora veamos cómo procesar y cargar una imagen en MATLAB. El siguiente bloque de código se muestra cómo cargar la imagen y el tamaño de la misma.

```
%# Leo el archivo
l = imread('fox_photo.jpg');
size(l)
```

```
>> 408  612  3

image(l)

%# Grafico para las primeras 300 columnas
% image(l(:, 1:300, :))

%Grafico para las primeras 300 filas
% image(l(1:300, :, :))
```

Se observa que la imagen tiene un tamaño de 408×612. El número 3 significa los canales de color: rojo, verde y azul (RGB, por su sigla en inglés). La imagen cargada se muestra en la figura 1. Dado que la imagen está definido por una cantidad de píxeles, podemos también graficarlo restringiendo las filas y columnas (ver código).

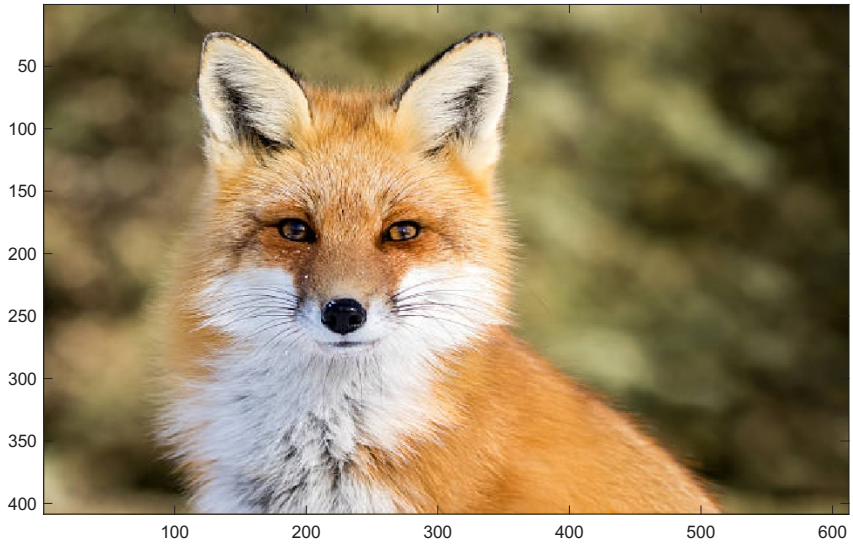


Figura 1. Imagen de un zorro cargado en MATLAB

También podemos graficar la misma imagen según el canal de color (figura 2).

```
%# Leo el archivo
l = imread('fox_photo.jpg');
% image(l)

%# Grafico para las primeras 300 columnas
% image(l(:, 1:300, :))

%Grafico para las primeras 300 filas
% image(l(1:300, :, :))

%# canales o colores
R = l(:, :, 1);
G = l(:, :, 2);
B = l(:, :, 3);

% image(B)

%imagen 200x200x3
% image(l(1:200, 1:200, :))

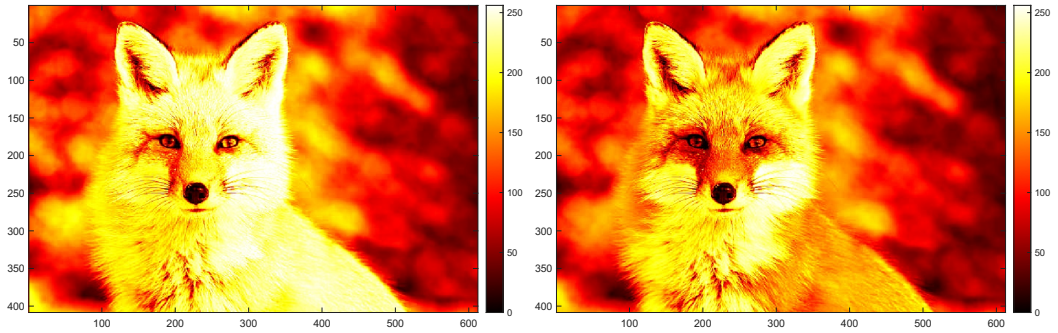
%# Agregamos una linea blanca
figure;
subplot(1, 2, 1);
```

```
l(:, 50:75, :) = 255;
image(l)
title('Franja blanca')
axis image;

%# Agregamos una linea negra
subplot(1, 2, 2);
l(150:175, :, :) = 0;
image(l)
title('Franja blanca y negra')
axis image;
```

Código 1: Procesamiento de imagen en MATLAB

Al igual que podemos graficar la imagen por color, podemos recortarla. Por ejemplo, en la figura 3 recortamos las primeras 200 filas y columnas de la imagen original.



(a) Para el canal rojo: `image(l(:, :, 1))` **(b)** Para el canal verde: `image(l(:, :, 2))`

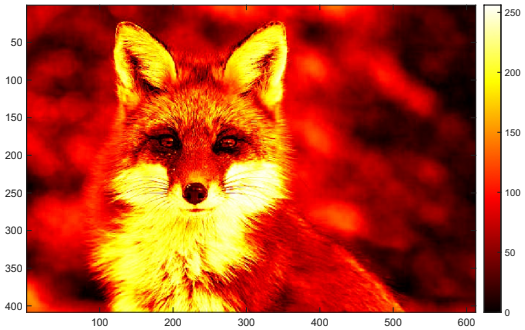


Figura 2. Para el canal azul: `image(l(:, :, 3))`

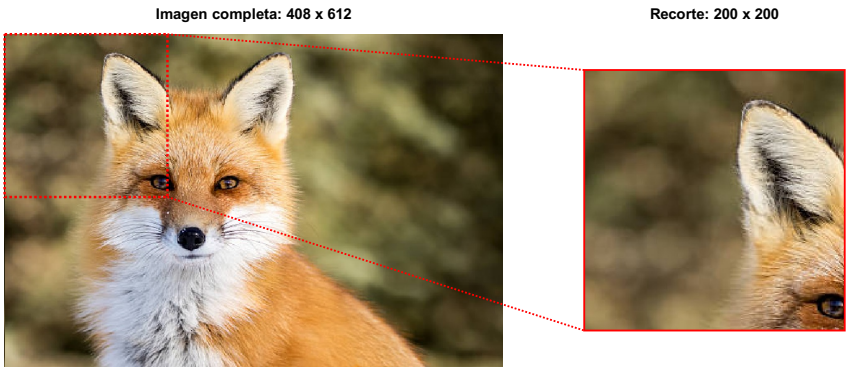


Figura 3. Imagen original recortada

Por último, también podemos agregar una línea blanca y negra (ver código 1). En la

figura 4 se representan las líneas blancas y negras.

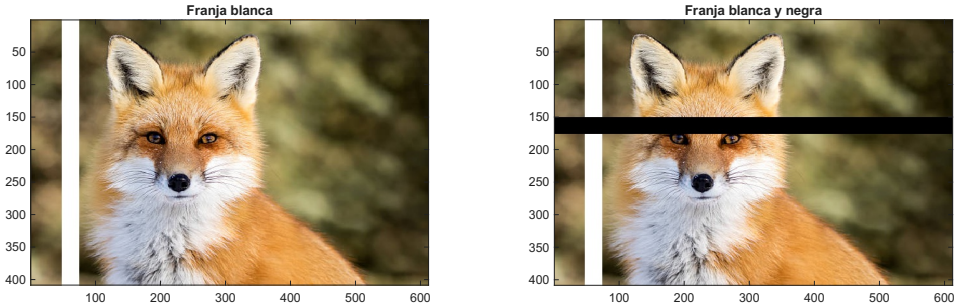


Figura 4. Franjas blanca y negra en la imagen original

Guía 18: Modelos avanzados de Simulink

Implementemos el siguiente sistema en MATLAB y Simulink:

$$G(s) = \frac{e^{-5s}}{s+2} \tag{1}$$

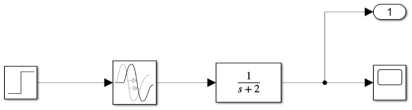
MATLAB

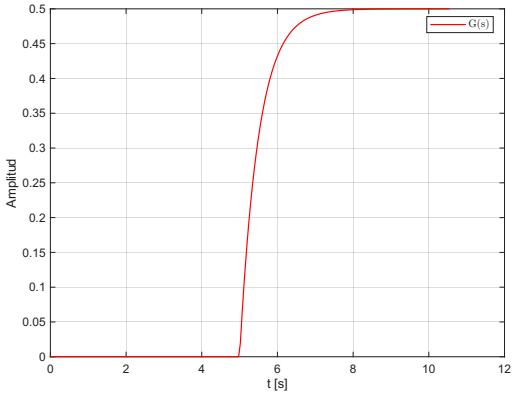
```
clc;
clear all;
close all;

% Funcion de transferencia
s = tf('s');
G = exp(-5*s)/(s + 2);

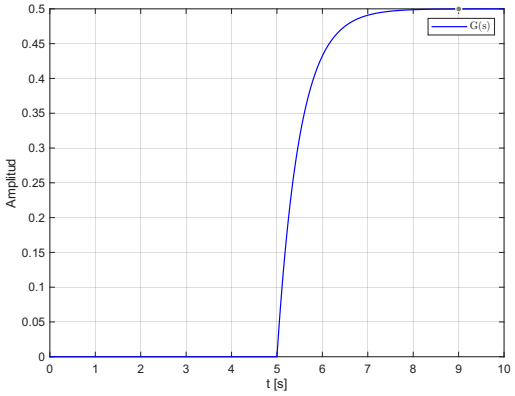
[yg, tg] = step(G);
```

Simulink





(a) Respuesta obtenida con MATLAB.



(b) Respuesta obtenida con Simulink.

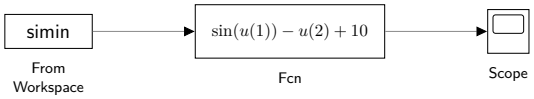
Figura 5. Comparación de la respuesta al escalón del sistema $G(s) = e^{-5s}/(s+2)$ mediante MATLAB y Simulink. Las respuestas son iguales

El siguiente código muestra cómo trabajar con funciones desde el *Workspace* de MATLAB para implementar en Simulink.

```
%Sistema desde el Workspace
t_array = 0:0.1:20;
x1 = t_array.*sin(sin(t_array)) + t_array;
x2 = t_array.^2.*exp(-0.5*t_array);

simin = [t_array', x1', x2'];
```

`simin` es una matriz de dimensión 201x3, que con el bloque *From Workspace* podemos cargarlo al Simulink. Además, podemos definir otro bloque, *Fcn*, que nos permite trabajar con las funciones ingresadas en el Workspace. El sistema es:



Y su respuesta

Las ventajas de representar funciones de transferencias desde el Workspace es que son fáciles de manipular y definir, permitiendo una comunicación más simple entre MATLAB y Simulink.

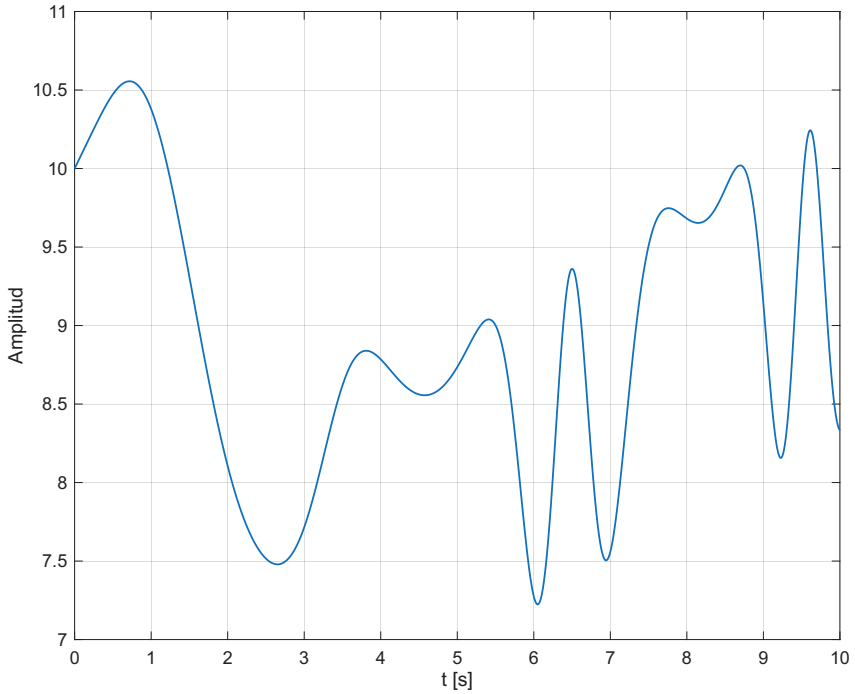


Figura 6. Salida de $\sin(u_1) - u_2 + 10$

Por último, definamos las siguientes funciones de transferencias:

$$G_1(s) = e^{-2s}\frac{3s}{s^2+s+2} \quad G_2(s) = \frac{1}{(s+2)(s+4)(s+5)} \quad G_3(s) = \begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases} \tag{2}$$

donde

$$A = \begin{bmatrix} -8 & -2{,}5 \\ 2 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$C = [0{,}5\ 0{,}75] \quad D = 0$$

Implementemos el siguiente sistema y veamos su respuesta (también su código a implementar).

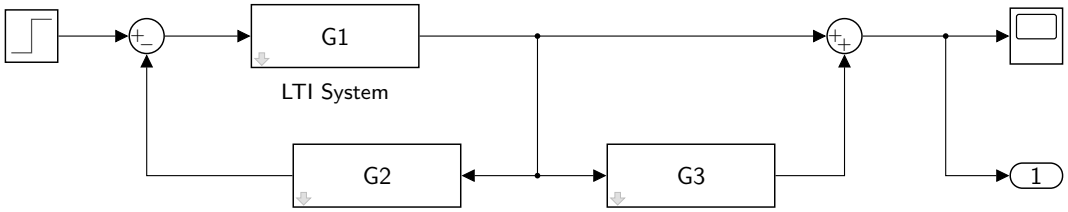


Figura 7. Diagrama de bloques con realimentación negativa mediante G_2.

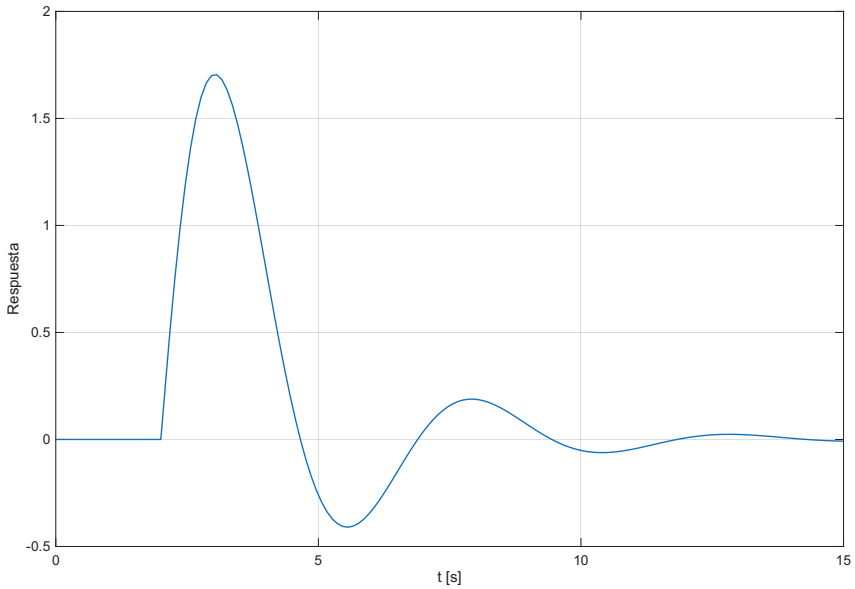


Figura 8. Respuesta al sistema de la figura de arriba

```
clc
clear all
close all

s = tf('s');
G1 = exp(-2*s)*3*s/(s^2 + s + 2);
G2 = zpk([], [-2, -4, -5], 1);

A = [-8, -2.5;
      2,    0];

B = [2;
     0];

C = [0.5, 0.75];
D = [0];

G3 = ss(A, B, C, D);

%salida
sis = sim('mod_18.slx');

%%
data = sis.yout.extractTimetable;
plot(sis.tout, data.Data)
```